

Mountain Lion Detection System

Jack Buzard

Max Conaway

Yad Galaly

Fernando Gonzalez

Rico Sandoval

Dylan Winn

System Overview

The Mountain Lion Detection System (MLDS) will be used by the San Diego County Parks and Recreation Department to detect mountain lions on land under their jurisdiction. The system will be based on another animal detection system created by the company “Animals-R-Here.” The system should be able to be reconfigured to fit the specific needs of the entity using it, in this case, possibly other parks in the State of California.

Purpose and Scope

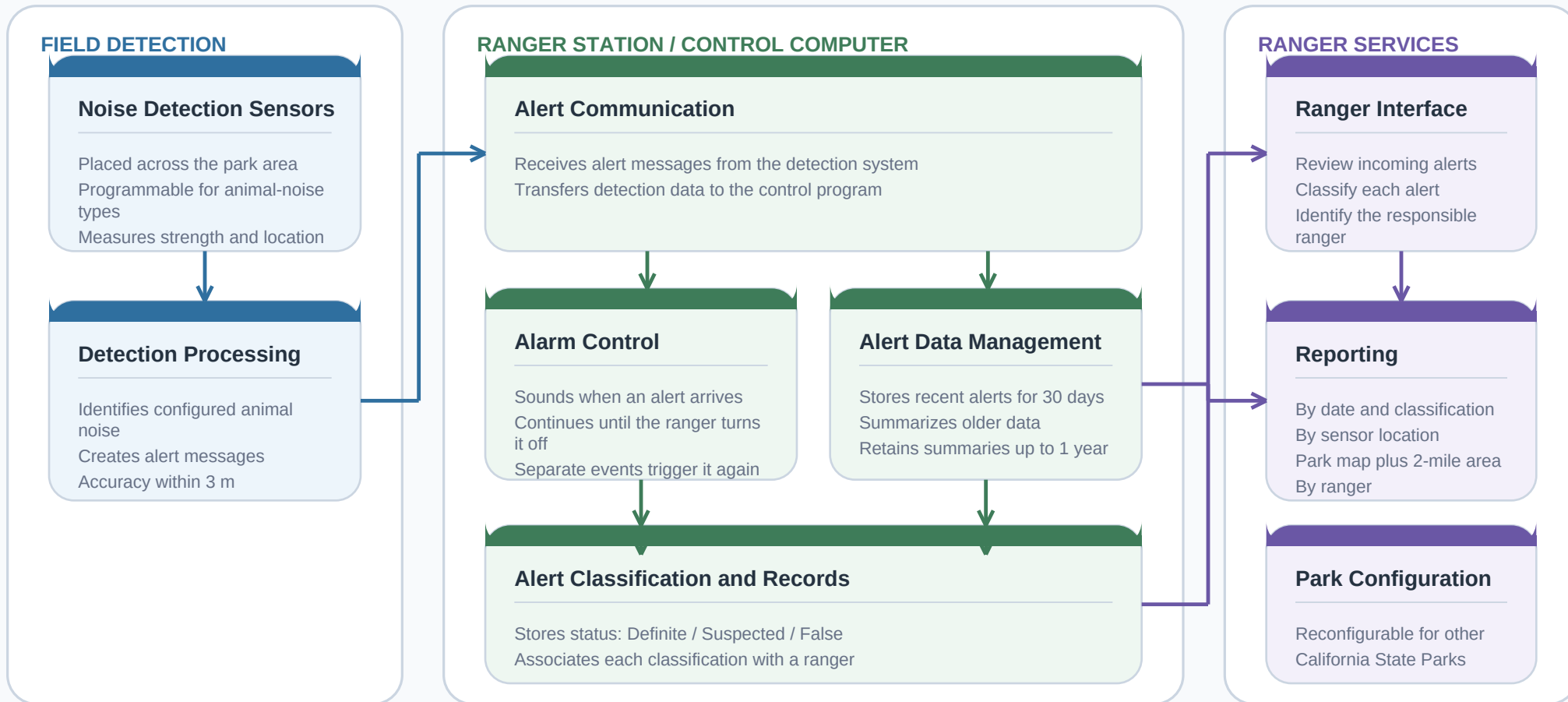
The purpose of the MLDS is to detect mountain lions on government land and to generate reports that government employees can use to make informed decisions regarding how to handle the animal. The system provides location data to support ranger decision-making. They can also use this information to analyze data on the local population to determine population size, territory, feeding habits, etc. The system will be able to detect other animals in the area as well.

System Context

The San Diego County Parks and Recreation Department will place audio sensors in locations in their parks. These sensors should work in all outdoor conditions. The sensors will send data to the controlling computer at a park ranger station. The controlling computer will then trigger an alarm that will continue until the ranger turns it off. The ranger on duty will be able to classify the alarm as definite, suspected, or false. The controlling computer will generate a report with relevant data that will be stored in the system in full for 30 days. It will generate a summary for all reports longer than 30 days but shorter than a year.

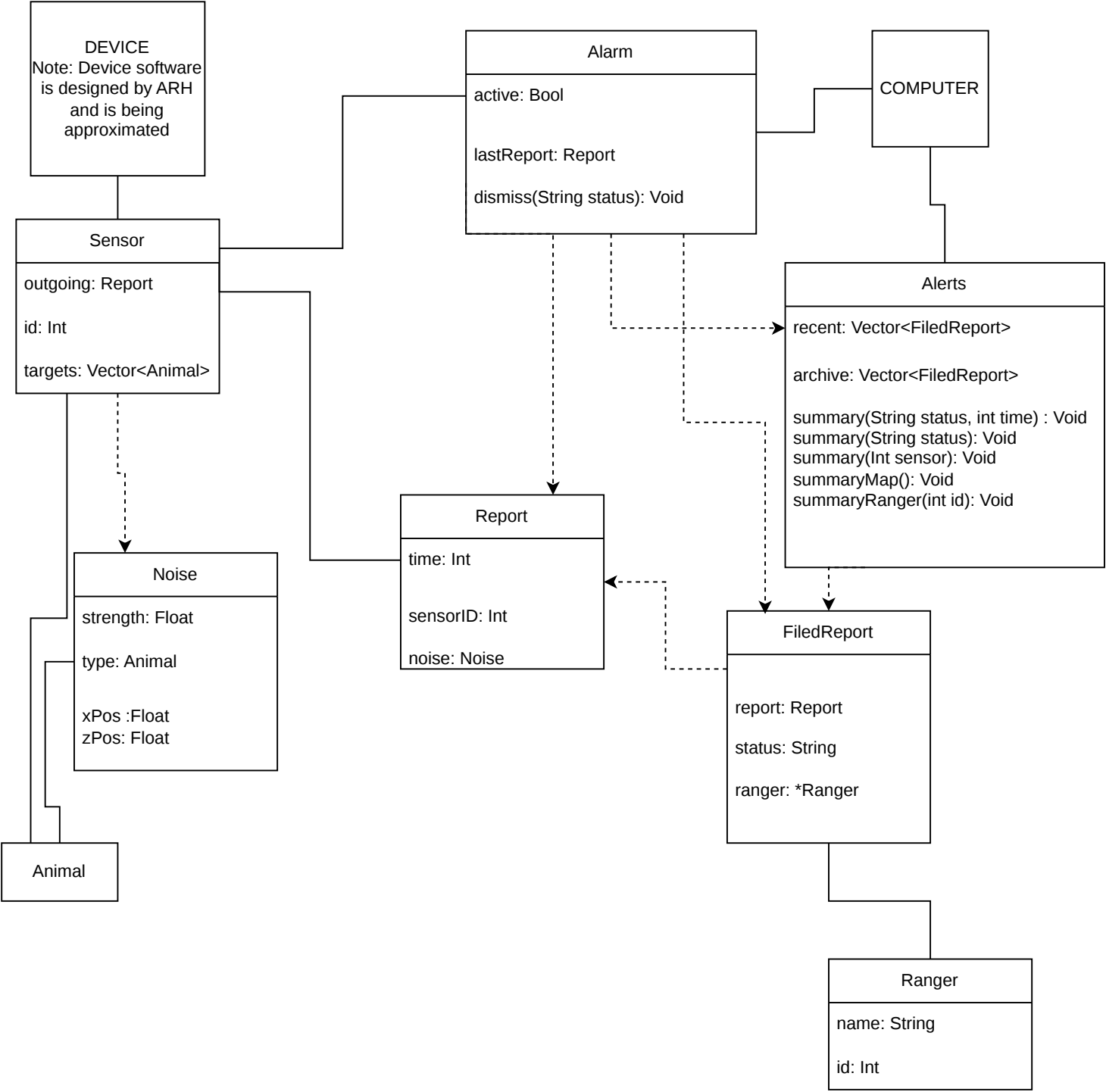
Mountain Lion Detection System

Software Architecture – Major Components and Data Flow



PRIMARY FLOW

Detect noise → Create alert → Control computer → Alarm / Store → Ranger classification → Reports



Description of Classes

Sensor

The Sensor class is an abstraction which represents the interface with the third-party hardware which will do the actual animal detection. Each instance of the class will represent a single physical sensor and contain a representation of the most recent data received from that sensor. The mechanism by which data is received is yet to be determined, but one potential option is to have the sensor push updates via a remote procedure call. When the sensors report potential animal sounds of interest, they will be represented by Noise objects, a list of which will exist inside each Sensor object. If a noise with sufficient strength is detected, the sensor will create a Report object referring to it and place it in its outgoing box.

Noise

The Noise class represents individual potential animal sounds reported by the sensors. These are ephemeral and will be erased and recreated each time new data comes in.

Animal

The Animal class enumerates different types of animals which the system is capable of detecting, and exists mainly to avoid the use of magic numbers.

Report

The Report class represents a detection event of sufficient interest to trigger an alarm, and will be persisted as part of a FiledReport.

Alarm

The Alarm class is a singleton which is responsible for monitoring the various sensors for outgoing reports. This will be accomplished by polling each sensor object on a regular basis, perhaps once per second. If a new report is found it will be wrapped in a FiledReport object and added to the list in the Alerts object.

FiledReport

The FiledReport class represents past alarms. It stores a copy of the original report along with how it has been classified and the Ranger who classified it.

Alerts

The Alerts class is a singleton which is responsible for storing and retrieving reports. The mechanism by which data is persisted has yet to be determined, but one potential option is to use an object relational model and an SQL database.

Ranger

The Ranger class represents the users of the system, for access control and record keeping purposes.

Description of Attributes

Mountain Lion Detection System

By: Fernando Gonzalez

Alerts

Stores every alert the controlling computer has received, split by age.

Attribute	Datatype	Description
recent	Vector<FiledReport>	Alerts received within the last 30 days, kept in full detail.
archive	Vector<FiledReport>	Alerts older than 30 days but received within the last year.

FiledReport

Pairs a received report with the ranger's classification of it.

Attribute	Datatype	Description
report	Report	The alert message received from the sensor.
status	String	The ranger's classification of the alert: "definite", "suspected", or "false".
ranger	*Ranger	Pointer to the ranger who classified the alert. Null until the alert is classified.

Report

The alert message sent by a sensor to the controlling computer.

Attribute	Datatype	Description
time	Int	Time the noise was detected or the alert was received (e.g., Unix timestamp in seconds).
sensorID	Int	The id of the Sensor that sent the alert.
noise	Noi	The detected noise: its type, strength, and location.

Noise

Describes a single detected animal noise.

Attribute	Datatype	Description
strength	Float	Strength of the detected noise.
type	Animal	The kind of animal the noise matches.
xPos	Float	East-west position of the noise, accurate to within 3 meters.
zPos	Float	North-south position of the noise, accurate to within 3 meters.

Sensor

A noise detection sensor placed in the park (approximates the Animals-R-Here device).

Attribute	Datatype	Description
outgoing	Report	The most recent report the sensor is sending to the controlling computer.
id	Int	Unique identifier of the sensor. Matches Report.sensorID.
targets	Vector<Animal>	Animal noise types this sensor is programmed to detect.

Animal

Represents a type of animal that a noise can match and that a sensor can be programmed to detect. This class has no attributes in the current design.

Alarm

Tracks the state of the alarm on the controlling computer.

Attribute	Datatype	Description
active	Bool	True while the alarm is sounding. Set to false when the ranger turns it off.
lastReport	Report	The report that most recently triggered the alarm. Used to decide whether a new noise is at a different location.

Ranger

A park ranger who uses the control program.

Attribute	Datatype	Description
name	String	Ranger's full name.
id	Int	Unique identifier of the ranger.

Description of Operations

Mountain Lion Detection System

By: Maximus Conaway

Alarm - dismiss(String status): Void

This operation allows the ranger to turn off an active alarm after an alert is received. The status parameter records the status connected to the alarm when it is dismissed. After the alarm is turned off, it will stay off until another separate noise is detected at a different location.

Alerts - summary(String status, int time): Void

This operation creates a summary of alerts using a classification and a certain time period. The status parameter can be definite, suspected, or false, and the time parameter is used to decide which alerts should be included in the summary.

Alerts - summary(String status): Void

This operation creates a summary based on the classification of the alerts. It allows the ranger to view alerts that were classified as definite, suspected, or false.

Alerts - summary(int sensor): Void

This operation creates a summary for a specific sensor. The sensor parameter identifies the sensor so the system can show the detections that came from that sensor location.

Alerts - summaryMap(): Void

This operation creates the information needed for a graphical map report. The map shows where detections happened in the park and in the areas within two miles of the park.

Alerts - summaryRanger(int id): Void

This operation creates a summary for a specific ranger. The id parameter identifies the ranger so the system can show the detection classifications made by that ranger.

How the Operations Work Together

When a sensor detects a programmed animal noise, the detection information is placed into a report and sent to the controlling computer. The computer uses the report to trigger the alarm and store the alert. The ranger can then turn off the alarm and classify the alert as definite, suspected, or false. The stored information can later be used by the Alerts operations to create summaries by classification, time, sensor, map location, or ranger.

Development plan

Task partitions

Rico Sandoval	Jack Buzzard	Dylan Winn	Max Conaway	Fernando Gonzalez	Yad Galaly
Title Page	UML Class DGRM	Description of Classes	Description of Operations	Description of Attributes.	Architectural Diagram
System Description					Development plan/timeline